

PL/SQL? 먹는건가요? - Part 1

은근한 비밀병기

박상윤

목차

1. PL/SQL 이란?
2. PL/SQL 종류 및 특징
3. 기본문법
 - 3.1.기본형태
 - 3.2.선언문
 - 3.3.반복문
 - 3.4.조건문

1. PL/SQL이란?

- Procedural Language extension to SQL 의 약자
- 여러 SQL을 하나의 블록(Block)으로 구성되어 SQL을 제어할 수 있다.
- 조건문(IF), 반복문(LOOP) 등 을 사용하여 유연한 프로그래밍이 가능하다.
- 커서(Cursor)를 사용하여 대용량 데이터를 처리할 때, 데이터를 분할하여 처리할 수 있다.
- 동적 SQL을 이용하여 문자열로 된 SQL을 실행할 수 있으며 DML, DDL, DCL을 사용할 수 있다.

2. PL/SQL 종류 및 특징 I

- 익명블록
 - 이름없는 코드들의 블록
 - 가장 기본적인 형태로 선언부, 실행부, 예외처리 부분이 있다
- 함수(Function)
 - 어떤 연산을 거치고 나서 단일한 값(single value)을 반환(return)하는 목적에 쓰인다
 - 반환값으로는 숫자, 날짜, 문자열 같은 기본 자료형이나, 배열이나 테이블 같은 집합의 형태로 제공될 수 있다
 - SQL 문 내에서 호출이 가능하다

2. PL/SQL 종류 및 특징II

- 프로시저(Procedure)
 - 함수와 비슷해 보이지만 SQL 구문 내에서 사용될 수 없다
 - 함수는 단일한 값을 반환하지만 프로시저는 다수의 값을 반환할 수 있다
 - IN, OUT, IN OUT 모드를 이용하여 요청값과 반환값을 구분한다
-> Postgresql에서는 OUT 모드는 지원하지 않는다
- 패키지(Package)
 - 함수, 프로시저, 변수 등을 개념적으로 연결하여 그룹으로 만들어진다
 - 사양(specification)을 기술하는 부분과 본체(body)로 나누어서 작성한다
 - 비즈니스 로직을 감출 수 있다

2. PL/SQL 종류 및 특징 III

- 트리거(Trigger)
 - 특정 이벤트가 발생했을때 자동으로 실행한다
 - enable 해두면 이벤트가 발생했을 때 자동으로 실행되며 명시적으로 실행시킬 수는 없다
 - 해당 이벤트에 연동하여 특정 처리를 하고 싶은 경우에 많이 쓰인다

3. 기본문법

3.1. 기본형태

- 여기서는 익명블록을 예로 든다.

```
do language plpgsql $$  
declare  
    {선언문}  
begin  
    {실행문}  
end;  
$$ ;
```

- \$\$~\$\$는 실제 구현하는 코드 블록이다. 이 안에 있는 내용들을 실행하게 된다.
- declare 에서 사용할 변수들을 먼저 지정한다. cursor 을 사용할 경우 여기서 선언한다 (cursor 은 나중에 별도로 언급예정)
- begin ~ end 에서 구현하고자 하는 코드를 작성한다. 조건문 및 반복문 등의 프로그래밍 요소도 사용가능하다.

3. 기본문법

3.1. 기본형태

```
DO language plpgsql $$
declare
    p_idx int4 := 225395 ;
    v_center_code text;
    v_idx int4;
begin
    select idx into v_idx
    from naver_kin sub
    LEFT JOIN
        common_code cc ON cc.code = sub.center_code
    where idx = p_idx;
    RAISE NOTICE 'idx : %',v_idx;
end;
$;
```

```
DO language plpgsql $$
declare
    p_idx int4 := 225395 ;
    v_center_code text;
    v_idx int4;
begin

    EXECUTE 'SELECT idx FROM naver_kin WHERE idx = $1'
    INTO v_idx
    USING p_idx;
    RAISE NOTICE 'idx : %',v_idx;
end;
$;
```


3. 기본문법

3.2. 선언문

- Declare~ Begin 사이에 선언한다.
- 사용할 변수들을 전부 미리 선언하도록 되어있다.
- 변수명 자료형 [:=기본값] 의 형태이다
- cursor 도 여기서 선언한다.

```
DECLARE
  v_next_idx int4;
  v_next_title varchar;
  v_next_created_at timestamp(6);
  v_prev_idx int4;
  v_prev_title varchar;
  v_prev_created_at timestamp(6);
BEGIN
```

3. 기본문법

3.3. 반복문(1) - LOOP

- 조건이 맞으면 반복되는 실행구문
- LOOP, FOR, WHILE 등이 있다.
- 기본적인 LOOP는 반복도중에 탈출할 조건에 만족하면 탈출하게 된다.
- LOOP ~ EXIT WHEN ~ END LOOP 의 구조로 한다.

```
DO $$
declare
  p_idx int4 := 5 ;
  idx int4 := 1;
  stars VARCHAR;
begin
  LOOP
    EXIT WHEN idx > 5;
    RAISE NOTICE '%', REPEAT('*', idx);
    idx := idx + 1;
  END LOOP;
end;
$$ language plpgsql;
```

3. 기본문법

3.3. 반복문(2) - WHILE

- WHILE 조건문 LOOP ~ END LOOP; 의 구조이다
- 기본 LOOP가 탈출할 조건을 이용한다고 하면 WHILE 은 실행할 조건을 이용한다.

```
DO $$
declare
  p_idx int4 := 5 ;
  idx int4 := 1;
  stars VARCHAR;
begin
  WHILE idx <= 5 LOOP

    RAISE NOTICE '%', REPEAT('*', idx);
    idx := idx + 1;
  END LOOP;
end ;
$$ language plpgsql;
```


3. 기본문법

3.3. 반복문(3) - FOR

- FOR 변수 IN 초기값..최대값 [BY 증가값] LOOP ~ END LOOP; 의 형태이다.
- 변수의 값은 초기값~최대값으로 증가한다. BY 가 있으면 해당 값 단위로 증가하며 없으면 1씩 증가한다.
- 최대값~초기값의 순서로 진행하고 싶으면 FOR 변수 IN REVERSE 최대값..초기값으로 한다.
- 배열이나 쿼리의 값을 받아서 처리도 가능하다.

3. 기본문법

3.3. 반복문(3) - FOR

```
DO $$
declare
    p_idx int4 := 5 ;
begin
    FOR i IN 1..p_idx LOOP
        RAISE NOTICE '%', REPEAT('*', i);
    END LOOP;
end ;
$$ language plpgsql;
```

```
----
DO $$
declare
    p_idx int4 := 5 ;
begin
    FOR i IN 1..p_idx BY 2 LOOP
        RAISE NOTICE '%', REPEAT('*', i);
    END LOOP;
end ;
$$ language plpgsql;
```

```
DO $$
declare
    p_idx int4 := 5 ;
begin
    FOR i IN REVERSE p_idx..1 LOOP
        RAISE NOTICE '%', REPEAT('*', i);
    END LOOP;
end ;
$$ language plpgsql;
```

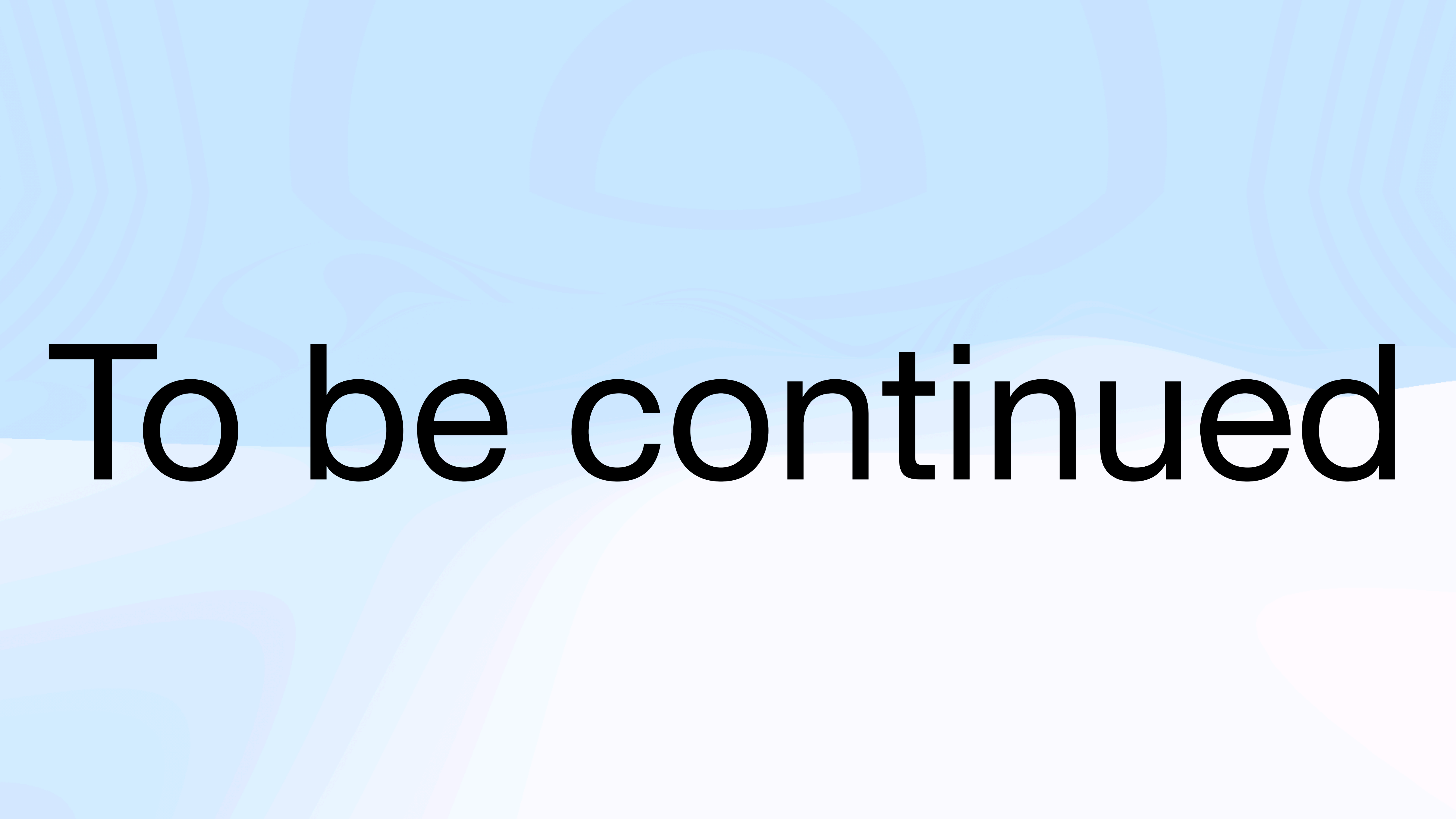
```
----
DO $$
declare
    res RECORD;
begin
    FOR res IN SELECT idx, approval_ip FROM approval_ip LOOP
        RAISE NOTICE ':%%', res.idx, res.approval_ip;
    END LOOP;
end ;
$$ language plpgsql;
```


3. 기본문법

3.4. 조건문

- IF 조건문 THEN 참의 실행구문 END IF; 가 가장기본 문법이다.
- 조건문은 일반 SQL 문의 WHERE 에 들어갈 수 있는 내용들도 가능하다.
- 거짓일때는 ELSE 구문을 사용하며 다중 IF 문을 사용할 경우 ELSEIF를 사용한다.
- 전체 구조는 다음과 같다.

```
DO $$
declare
  a integer := 20;
  b integer := 40;
BEGIN
  IF a > b THEN
    RAISE NOTICE 'a가 b보다 더 큼니다.';
  ELSEIF a = b THEN
    RAISE NOTICE 'a와 b가 같습니다.';
  ELSE
    RAISE NOTICE 'a와 b가 같지 않고, a가 b보다 더 크지도 않습니다.';
  END IF;
end;
$$ language plpgsql;
```



To be continued

Coming Up Next
Function